Empty-Shelf Detector — Week 1

Fine-tune YOLO11 to flag out-of-stocks on retail shelf photos.

0. Turn the GPU on first

Colab gives you a free GPU but **not by default.** Do this once:

Runtime → Change runtime type → Hardware accelerator → T4 GPU → Save.

Then run the cell below. If it prints a table with "Tesla T4", you're set. If it errors, the GPU isn't on yet.

```
!nvidia-smi
```

```
Wed Jun 24 16:18:01 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   41C    P8             11W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

1. Install

`ultralytics` is the YOLO library. `roboflow` is just to pull the dataset.

```
!pip install ultralytics roboflow -q

import ultralytics
ultralytics.checks()   # prints versions + confirms the GPU is visible to YOLO
from ultralytics import YOLO
```

```
Ultralytics 8.4.76 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Setup complete ✅ (2 CPUs, 12.7 GB RAM, 47.3/112.6 GB disk)
```

2. Get the data

We use a ready-labeled empty-shelf dataset from **Roboflow Universe** so you skip annotation entirely this week. A good candidate:

- `fyp-ormnr/supermarket-empty-shelf-detector` (≈500 images, labels empty / out-of-stock regions)

Other options if that one's down: `empty-space-detection-capstone/out-of-stock-detection`, or browse https://universe.roboflow.com/browse/retail

How to get YOUR download snippet (60 seconds)

1. Open the dataset page on Roboflow Universe.
2. Click **Download Dataset** → format **YOLOv11** → **show download code**.
3. It hands you a snippet with *your* API key + the exact workspace/project/version filled in.
4. Paste that over the placeholder cell below.

A free Roboflow account gives you the API key. The snippet looks like the template below — replace the three ALL-CAPS values (or just paste Roboflow's generated version verbatim).

```
# ── Replace with the snippet from Roboflow's "show download code" ──
# !pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_API_KEY")
project = rf.workspace("fyp-ormnr").project("supermarket-empty-shelf-detector")
version = project.version(4)
dataset = version.download("yolov11")

# ────────────────────────────────────────────────────────────────

print("Downloaded to:", dataset.location)
```

```
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Supermarket-Empty-Shelf-Detector--4 to yolov11:: 100%|██████████| 45056/45056 [00:

Extracting Dataset Version Zip to Supermarket-Empty-Shelf-Detector--4 in yolov11:: 100%|██████████| 1006/1006 [00:00<
```

Quick fix Roboflow's data.yaml (do this — it saves an annoying error)

Roboflow writes relative paths into `data.yaml` that sometimes break in Colab. This rewrites them to absolute paths so training just works.

```
import yaml, os

base = "/content/Supermarket-Empty-Shelf-Detector--4"   # the path from your error
yaml_path = os.path.join(base, "data.yaml")

with open(yaml_path) as f:
    d = yaml.safe_load(f)

def find_images(base, names):
    for n in names:
        p = os.path.join(base, n, "images")
        if os.path.isdir(p):
            return p
    return None

d["train"] = find_images(base, ["train"])
d["val"]   = find_images(base, ["valid", "val"])   # Roboflow uses "valid"
test = find_images(base, ["test"])
if test:
    d["test"] = test

with open(yaml_path, "w") as f:
    yaml.safe_dump(d, f)

print("Classes:", d.get("names"))
print("train:", d["train"])
print("val:  ", d["val"])
```

```
Classes: ['100- O-O-S']
train: /content/Supermarket-Empty-Shelf-Detector--4/train/images
val:   /content/Supermarket-Empty-Shelf-Detector--4/valid/images
```

3. Fine-tune YOLO11

`yolo11n` = the nano model: smallest and fastest, perfect for a free T4 and a small dataset. (`yolo11s`/`yolo11m` are bigger and more accurate but slower — try them later.)

You're not training from scratch — you start from weights pretrained on millions of images and *fine-tune* on shelves. That's why ~50 epochs on a few hundred images is enough to get something real.

Why YOLO11 and not the newer YOLO26? YOLO26 shipped Jan 2026 and is great, but YOLO11 has a year of tutorials, forum answers, and Stack Overflow threads behind it. When you hit an error at 11pm, you want the version everyone else has already debugged. Switching later is literally one word: `yolo26n.pt`. Learn on the well-worn path.

```
# model = YOLO("/content/drive/MyDrive/shelf_project/best.pt")         # pretrained weights -> trained with Robotfl
model = YOLO("yolo11n.pt")
results = model.train(
    data=yaml_path,
    epochs=50,
    imgsz=640,
    batch=16,
    patience=15,                    # stop early if it plateaus
    name="shelf_v1",
)
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      42/50      3.05G      1.006     0.7417     0.9563         67        640: 100% ━━━━━━━━━━━━ 22/22 3.5it/s 6.3s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 3.4
                   all         97        600       0.88      0.753      0.874       0.59

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      43/50      3.05G     0.9753     0.7144     0.9538         75        640: 100% ━━━━━━━━━━━━ 22/22 4.3it/s 5.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 3.2
                   all         97        600      0.877      0.783      0.878      0.597

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      44/50      3.05G     0.9672     0.7028     0.9491         67        640: 100% ━━━━━━━━━━━━ 22/22 3.6it/s 6.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 3.1
                   all         97        600      0.851      0.768       0.86      0.573

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      45/50      3.05G     0.9511     0.6907     0.9335         71        640: 100% ━━━━━━━━━━━━ 22/22 4.2it/s 5.2s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 3.4
                   all         97        600      0.919      0.755      0.871      0.598

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      46/50      3.05G     0.9301     0.6626      0.925         55        640: 100% ━━━━━━━━━━━━ 22/22 3.6it/s 6.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 4.8
                   all         97        600      0.909       0.78      0.878      0.607

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      47/50      3.05G     0.9465     0.6639     0.9392         66        640: 100% ━━━━━━━━━━━━ 22/22 3.4it/s 6.4s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 1.6
                   all         97        600      0.871      0.802      0.874      0.606

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      48/50      3.05G     0.9035     0.6504      0.922         69        640: 100% ━━━━━━━━━━━━ 22/22 3.1it/s 7.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 4.0
                   all         97        600       0.88      0.796      0.875      0.602

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      49/50      3.05G     0.9025     0.6381     0.9263         70        640: 100% ━━━━━━━━━━━━ 22/22 3.6it/s 6.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 4.2
                   all         97        600      0.869      0.812      0.881      0.609

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      50/50      3.05G     0.9107     0.6343      0.923         71        640: 100% ━━━━━━━━━━━━ 22/22 3.5it/s 6.3s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 2.4
                   all         97        600      0.878      0.803      0.883      0.615

50 epochs completed in 0.120 hours.
Optimizer stripped from /content/runs/detect/shelf_v1/weights/last.pt, 5.5MB
Optimizer stripped from /content/runs/detect/shelf_v1/weights/best.pt, 5.5MB

Validating /content/runs/detect/shelf_v1/weights/best.pt...
Ultralytics 8.4.76 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 1.6
                   all         97        600       0.88      0.806      0.884      0.616
Speed: 0.3ms preprocess, 3.4ms inference, 0.0ms loss, 4.0ms postprocess per image
Results saved to /content/runs/detect/shelf_v1
```

4. How good is it?

`mAP50` is the headline number (mean average precision at IoU 0.5). Don't chase a specific value yet — just read it, eyeball the predictions next, and note it in your writeup. The training run also saved plots (confusion matrix, PR curve) into the run folder.

```
metrics = model.val()
print("mAP50:    ", round(metrics.box.map50, 3))
print("mAP50-95: ", round(metrics.box.map, 3))
```

```
Ultralytics 8.4.76 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2527.2±649.9 MB/s, size: 90.0 KB)
val: Scanning /content/Supermarket-Empty-Shelf-Detector--4/valid/labels.cache... 97 images, 0 backgrounds, 0 corrupt:
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 7/7 1.6it
                   all         97        600       0.88      0.805      0.884      0.615
Speed: 8.6ms preprocess, 8.0ms inference, 0.0ms loss, 4.5ms postprocess per image
Results saved to /content/runs/detect/val
mAP50:     0.884
mAP50-95:  0.615
```

5. Run it on an image

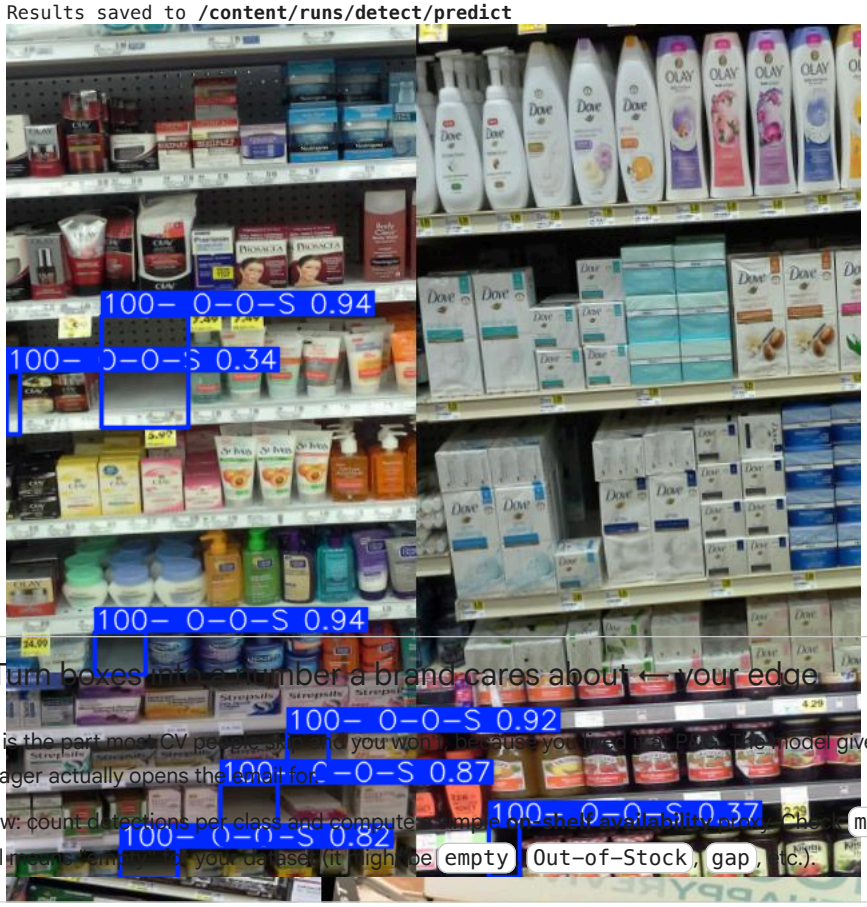
This runs your trained model on validation images and shows the boxes. Later, swap in a photo *you* took at a real store — that's step 7.

```
import glob
from PIL import Image
from IPython.display import display

val_images = glob.glob(os.path.join(dataset.location, "valid", "images", "*"))
val_images = val_images or glob.glob(os.path.join(dataset.location, "val", "images", "*"))

preds = model.predict(val_images[:3], save=True, conf=0.25)
for p in preds:
    display(Image.fromarray(p.plot()[..., ::-1]))   # BGR->RGB
```

```
0: 640x640 7 100- 0-0-Ss, 16.8ms
1: 640x640 11 100- 0-0-Ss, 16.8ms
2: 640x640 9 100- 0-0-Ss, 16.8ms
Speed: 1.4ms preprocess, 16.8ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)
Results saved to /content/runs/detect/predict
```



6. Turn boxes into a number a brand cares about ← your edge

This is the part most CV people skip. And you won't be because you're [illegible] model gives boxes; *you* know which metric a brand manager actually opens the email for.

Below: count detections per class and compute a simple **on-shelf availability** proxy. Check `model.names` first and set which class label means "empty" in your dataset (it might be `empty`, `Out-of-Stock`, `gap`, etc.).

```
print("Class names in this model:", model.names)

# ⚠ set this to whichever label means an empty/out-of-stock region
EMPTY_LABELS = {"empty", "Empty", "Out-of-Stock", "out-of-stock", "gap", '100- 0-0-S'}

def shelf_report(image_path, conf=0.25):
    r = model.predict(image_path, conf=conf, verbose=False)[0]
    h, w = r.orig_shape          # image size in pixels
    n_gaps = len(r.boxes)        # number of out-of-stock regions

    gap_area = sum(bw * bh for _, _, bw, bh in r.boxes.xywh.tolist())
    gap_pct = gap_area / (h * w) * 100

    print(f"\n{os.path.basename(image_path)}")
    print(f"  out-of-stock regions: {n_gaps}")
    print(f"  empty area: {gap_pct:.1f}% of the image")
    return n_gaps, gap_pct

for img in val_images[:3]:
    shelf_report(img)
```

```
Class names in this model: {0: '100- 0-0-S'}

test2020_20_jpg.rf.26bca2cf4[illegible]2ad0081c659.jpg
  out-of-stock regions: 7
  empty area: 3.0% of the image

test2020_1071_jpg.rf.ce590da92ef8f7db[illegible]3576ch.jpg
  out-of-stock regions: 11
  empty area: 7.9% of the image

test2020_121_jpg.rf.5344c411b[illegible]bf0c0b.jpg
  out-of-stock regions: 9
  empty area: 3.0% of the image
```

7. Save the model + ship it

Your trained weights are at `runs/detect/shelf_v1/weights/best.pt`. The cell downloads them.

Then this is what makes it a real project, not a tutorial you watched:

1. Take 3–5 photos of an actual shelf at a local store (some full, some with gaps).
2. Run them through `shelf_report()` above.
3. Push to a GitHub repo: the notebook, a few result images, and a short README — what it does, the mAP50, the empty area metric, and one line connecting it to retail out-of-stocks (you know that pitch cold).

That repo is the artifact. It's your first CV project *and* the first brick of a domain-specific portfolio. Expansion comes after the ugly working version exists — not before.

```
from google.colab import files

# best.pt is the model. Later run with YOLO("best.pt")
candidates = glob.glob("/content/runs/detect/**/weights/best.pt", recursive=True)
print("Found:", candidates)

latest = max(candidates, key=os.path.getmtime)   # the most recent run
print("Downloading:", latest)
files.download(latest)
```

```
Found: ['/content/runs/detect/shelf_v1/weights/best.pt']
Downloading: /content/runs/detect/shelf_v1/weights/best.pt
```

```
import shutil

# 1. create the Drive folder (this is the part that didn't exist)
os.makedirs("/content/drive/MyDrive/shelf_project", exist_ok=True)

# 2. find the best.pt your training produced
src = max(glob.glob("/content/runs/detect/**/weights/best.pt", recursive=True),
          key=os.path.getmtime)
print("Found weights at:", src)

# 3. copy it into Drive so it survives future sessions
dst = "/content/drive/MyDrive/shelf_project/best.pt"
shutil.copy(src, dst)
print("Saved to:", dst)

# 4. now this works
model = YOLO(dst)
print("Loaded ✓")
```

```
Found weights at: /content/runs/detect/shelf_v1/weights/best.pt
Saved to: /content/drive/MyDrive/shelf_project/best.pt
Loaded ✓
```

8. Try it with my own data!

Pictures taken from the stores already! Let's try it with some fresh data.

1. Make a photo folder in Drive and upload into it
2. Call the shelf_report function from step 6 (the single-class version) and run it on every photo
3. See the boxes on my photos

```
# 1. Make a photo folder in Drive and upload into

!pip install pillow-heif -q

from PIL import Image
import pillow_heif

pillow_heif.register_heif_opener()      # teaches PIL to open .HEIC

photo_dir = "/content/drive/MyDrive/shelf_project/photos"
os.makedirs(photo_dir, exist_ok=True)

heic_files = glob.glob("/content/*.HEIC") + glob.glob("/content/*.heic")
print("Found", len(heic_files), "HEIC files")

for path in heic_files:
    img = Image.open(path).convert("RGB")
    name = os.path.splitext(os.path.basename(path))[0] + ".jpg"
    out = os.path.join(photo_dir, name)
    img.save(out, "JPEG", quality=90)
    print("Converted ->", out)
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.7/5.7 MB 117.3 MB/s eta 0:00:00
Found 0 HEIC files
```

```
# 2. Call the shelf_report function from step 6 (the single-class version) and run it on every photo

for img in glob.glob(f"{photo_dir}/*"):
  shelf_report(img)
```

```
# 3. See the boxes on my photos
for img in glob.glob(f"{photo_dir}/*"):
    r = model.predict(img, conf=0.25, verbose=False)[0]
    display(Image.fromarray(r.plot()[..., ::-1]))   # photo with boxes drawn
```

Results: Save it and upload it to Github

```
# 1. Save the annotated photos as files
results_dir = "/content/drive/MyDrive/shelf_project/results"
os.makedirs(results_dir, exist_ok=True)

for img in glob.glob(f"{photo_dir}/*.jpg"):
    r = model.predict(img, conf=0.25, verbose=False)[0]
    out = os.path.join(results_dir, "pred_" + os.path.basename(img))
    Image.fromarray(r.plot()[..., ::-1]).save(out)
    print("Saved", out)
```

Train with the photos I take from different stores (71 images)

```
project_2 = rf.workspace("ash-feng").project("empty-shelf-detector-18ba1")
dataset_2 = project_2.version(7).download("yolov11")


# Load YOUR trained model, not the generic one
model_2 = YOLO("/content/drive/MyDrive/shelf_project/best.pt")

# Continue training on your 102 photos
results_2 = model_2.train(
    data=f"{dataset_2.location}/data.yaml",  # your v3 dataset
    epochs=50,
    imgsz=640,        # stop early if no improvement for 15 epochs
    name="finetune_v1.5"
)
```

```
loading Roboflow workspace...
loading Roboflow project...
Exporting format yolov11 in progress : 0.1%
Version export complete for yolov11 format
Downloading Dataset Version Zip in Empty-Shelf-Detector-7 to yolov11:: 100%|██████████| 21826/21826 [00:02<00:00, 7
```

```
Extracting Dataset Version Zip to Empty-Shelf-Detector-7 in yolov11:: 100%|██████████| 493/493 [00:00<00:00, 5094.6(
Ultralytics 8.4.76 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr=0.0,

                   from  n    params  module                                       arguments
  0                  -1  1       464  ultralytics.nn.modules.conv.Conv             [3, 16, 3, 2]
  1                  -1  1      4672  ultralytics.nn.modules.conv.Conv             [16, 32, 3, 2]
  2                  -1  1      6640  ultralytics.nn.modules.block.C3k2            [32, 64, 1, False, 0.25]
  3                  -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
  4                  -1  1     26080  ultralytics.nn.modules.block.C3k2            [64, 128, 1, False, 0.25]
  5                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
  6                  -1  1     87040  ultralytics.nn.modules.block.C3k2            [128, 128, 1, True]
  7                  -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  8                  -1  1    346112  ultralytics.nn.modules.block.C3k2            [256, 256, 1, True]
  9                  -1  1    164608  ultralytics.nn.modules.block.SPPF            [256, 256, 5]
 10                  -1  1    249728  ultralytics.nn.modules.block.C2PSA           [256, 256, 1]
 11                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 12             [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 13                  -1  1    111296  ultralytics.nn.modules.block.C3k2            [384, 128, 1, False]
 14                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 15             [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 16                  -1  1     32096  ultralytics.nn.modules.block.C3k2            [256, 64, 1, False]
 17                  -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
 18            [-1, 13]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 19                  -1  1     86720  ultralytics.nn.modules.block.C3k2            [192, 128, 1, False]
 20                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 21            [-1, 10]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 22                  -1  1    378880  ultralytics.nn.modules.block.C3k2            [384, 256, 1, True]
 23        [16, 19, 22]  1    430867  ultralytics.nn.modules.head.Detect           [1, 16, None, [64, 128, 256]]
YOLO11n summary: 182 layers, 2,590,035 parameters, 2,590,019 gradients, 6.4 GFLOPs

Transferred 499/499 items from pretrained weights
Freezing layer 'model.23.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed ✅
train: Fast image access ✅ (ping: 0.0±0.0 ms, read: 1918.1±861.9 MB/s, size: 85.7 KB)
train: Scanning /content/Empty-Shelf-Detector-7/train/labels... 213 images, 3 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━
train: New cache created: /content/Empty-Shelf-Detector-7/train/labels.cache
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weigh
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 631.8±487.0 MB/s, size: 99.8 KB)
val: Scanning /content/Empty-Shelf-Detector-7/valid/labels... 21 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━
val: New cache created: /content/Empty-Shelf-Detector-7/valid/labels.cache
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0'
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bia
Plotting labels to /content/runs/detect/finetune_v1.5-6/labels.jpg...
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/runs/detect/finetune_v1.5-6
Starting training for 50 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
```

```
from ultralytics import YOLO
import glob
m = YOLO("/content/runs/detect/finetune_v1.5-3/weights/best.pt")
m.predict(sorted(glob.glob(f"{dataset_2.location}/valid/images/*.jpg")), save=True, conf=0.4)
```

```
0: 640x640 6 empty shelf gaps, 4.7ms
1: 640x640 4 empty shelf gaps, 4.7ms
2: 640x640 (no detections), 4.7ms
3: 640x640 2 empty shelf gaps, 4.7ms
4: 640x640 3 empty shelf gaps, 4.7ms
5: 640x640 1 empty shelf gap, 4.7ms
6: 640x640 1 empty shelf gap, 4.7ms
7: 640x640 4 empty shelf gaps, 4.7ms
8: 640x640 4 empty shelf gaps, 4.7ms
9: 640x640 (no detections), 4.7ms
10: 640x640 2 empty shelf gaps, 4.7ms
11: 640x640 2 empty shelf gaps, 4.7ms
12: 640x640 6 empty shelf gaps, 4.7ms
13: 640x640 6 empty shelf gaps, 4.7ms
14: 640x640 2 empty shelf gaps, 4.7ms
15: 640x640 4 empty shelf gaps, 4.7ms
16: 640x640 4 empty shelf gaps, 4.7ms
17: 640x640 1 empty shelf gap, 4.7ms
18: 640x640 7 empty shelf gaps, 4.7ms
19: 640x640 (no detections), 4.7ms
20: 640x640 10 empty shelf gaps, 4.7ms
Speed: 2.8ms preprocess, 4.7ms inference, 0.7ms postprocess per image at shape (1, 3, 640, 640)
Results saved to /content/runs/detect/predict-6
```

```
[ultralytics.engine.results.Results object with attributes:

 boxes: ultralytics.engine.results.Boxes object
 keypoints: None
 masks: None
 names: {0: 'empty shelf gap'}
 obb: None
 orig_img: array([[[151, 225, 227],
         [151, 225, 227],
         [151, 225, 227],
         ...,
         [135, 131, 120],
         [159, 153, 142],
         [219, 213, 202]],

        [[151, 225, 227],
         [151, 225, 227],
         [151, 225, 227],
         ...,
         [174, 170, 159],
         [181, 175, 164],
         [211, 205, 194]],

        [[151, 225, 227],
         [151, 225, 227],
         [151, 225, 227],
         ...,
         [252, 248, 237],
         [255, 253, 242],
         [255, 255, 247]],

        ...,
```

```
import IPython.display as ipd, glob, os
latest = sorted(glob.glob("/content/runs/detect/predict*"), key=os.path.getmtime)[-1]
for p in sorted(glob.glob(f"{latest}/*.jpg"))[:-1]:
    print(os.path.basename(p)); ipd.display(ipd.Image(filename=p))
```